

```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.P05.Boot
{
    public static class SceneIds
    {
        {
            public const string Boot = "S00_Boot";
            public const string Main = "S05_Main";
        }
        /// <summary>
        /// S00_Boot      Mock → →      T001④
        ///      Boot      UI T001b  Boot      =
        /// </summary>
        public sealed class BootRunner : MonoBehaviour
        {
            IEnumerator Start()
            {
                BootServices.RegisterAll();
                var sequence = new BootSequence(SwitchToMain);
                if (!sequence.RunAll())
                {
                    Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                    yield break;
                }
                yield return null;
            }
            bool SwitchToMain()
            {
                StartCoroutine(LoadMainRoutine());
                return true;
            }
            IEnumerator LoadMainRoutine()
            {
                {
                    var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
                    while (op != null && !op.isDone) yield return null;
                }
            }
        }
    }
}

// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.P05.Boot
{
    /// <summary>
```

```

/// Boot      T001
/// ①Logger→② →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦      [BOOT] n/7
/// ⑦      Play  =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
/// X006      SY MOCK
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &= Run(1, "Logger", LoggerStep);
        ok &= Run(2, "Time", TimeStep);
        ok &= Run(3, "Config", ConfigStep);
        ok &= Run(4, "Save", SaveStep);
        ok &= Run(5, "ILocale", LocaleStep);
        ok &= Run(6, "Ads.Preload", AdsStep);
        ok &= Run(7, "SwitchMain", () =&gt; _switchScene != null && _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; //      JS sink platform T001c
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 && !time.TimeCheated();
    }
}

```

```

        static bool ConfigStep()
        {
            var config = ServiceLocator.Get<IConfigService>();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock= X009 manifest
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get<ISaveService>();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get<ILocaleService>();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); // +
        }
        static bool AdsStep()
        {
            ServiceLocator.Get<IAdsService>().Preload("boot_warmup");
            return true;
        }
    }
}

// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P05.Boot
{
    /// <summary>
    /// Mock SY MOCK X006 Mock X002a AppID touristappid
    /// T001c WX/TT Channel Editor Boot
    /// </summary>
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet<ITimeService>(out _)) return; //
            ServiceLocator.Register<ITimeService>(new MockTimeService());
            ServiceLocator.Register<IConfigService>(new MockConfigService());
            ServiceLocator.Register<ISaveService>(new MockSaveService());
            ServiceLocator.Register<ILocaleService>(new MockLocaleService());
            ServiceLocator.Register<IAdsService>(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY MOCK)");
        }
    }
}

```

```

}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source => TimeSource.Mock;
    public long ClientNowMs() => DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// <summary>SY MOCK X006 TCB serverTime→HTTPS Date → T001b </summary>
    public long ServerNowMs() => ClientNowMs();
    public bool TimeCheated() => Math.Abs(ServerNowMs() - ClientNowMs()) > 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable<T>(string tableId) where T : class
        => throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable<T>(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        => new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) => "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) => PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) => PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// <summary>X006 T001b v1→v2 </summary>
    public SyncResult CloudSync() => new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// <summary>key key T001b cfg_i18n + NUnit </summary>
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        => new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action<AdLifecycle> OnAdEvent;
    public Task<RewardResult> ShowRewarded(string pointId)
        => Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
}

```

```

        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}

// MainUI.cs
using UnityEngine;
using UnityEngine.SceneManagement;
namespace Sy.P05.Newbie
{
    public static class NewbieSceneIds
    {
        public const string Main = "S05_Main";
        public const string Battle = "S05_Battle";
    }
    /// <summary>S05          →S05_Battle M5          —          </summary>
    public sealed class MainUI : MonoBehaviour
    {
        public void StartSurvival()
        {
            SceneManager.LoadScene(NewbieSceneIds.Battle);
        }
        void Awake()
        {
            Application.targetFrameRate = 30;
            Sy.Core.MainArtInjector.Apply(transform, "P05_scene_01", "P05_general_01_zhaoyun_awake");
        }
    }
}

// BattleLauncher.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P05.Newbie
{
    /// <summary>
    /// S05_Battle          FurnaceSim+TutorialFlow+FurnaceView    Tick
    ///    Sy.Telemetry Editor =Debug sink T001e          +TCB
    /// HUD          6
    /// </summary>
    public sealed class BattleLauncher : MonoBehaviour
    {
        FurnaceSim _sim;
        TutorialFlow _tutorial;
        FurnaceView _view;
        Sy.Telemetry.TelemetryService _telemetry;
        int _lastCoins = -1, _lastCitizens = -1, _lastWood = -1, _lastFuel = -1;
        void Awake()
        {

```

```
Application.targetFrameRate = 30;
var canvas = FindObjectOfType<Canvas>();
_sim = new FurnaceSim();
_view = gameObject.AddComponent<FurnaceView>();
_view.Initialize(canvas, _sim);
_telemetry = new Sy.Telemetry.TelemetryService();
_telemetry.AddSink(new TelemetryDebugSink());
_tutorial = new TutorialFlow(
    _sim,
    hint => {
        Debug.Log("[HINT] " + hint);
        _view.SetHint(hint);
    },
    step => { _telemetry.Track(Sy.Telemetry.EventIds.NewbieStep,
        new Dictionary<string, string> { { "step", step.ToString() } } });
    _view.ClaimRequested += OnClaimRequested;
    _view.FirewoodRequested += OnFirewoodRequested;
    _view.BuildRequested += OnBuildRequested;
    _view.DigRequested += OnDigRequested;
    _view.NearDispatchRequested += () => OnDispatchRequested(0);
    _view.FarDispatchRequested += () => OnDispatchRequested(1);
    _view.SocialTapped += OnSocialTapped;
    _sim.PhaseChanged += OnPhaseChanged;
    _sim.DigStageReached += OnDigStageReached;
}
void Update()
{
    _sim.Tick(Time.deltaTime);
    _view.SyncView(_sim);
    if (_sim.Coins != _lastCoins) { _lastCoins = _sim.Coins; _view.SetCoins(_lastCoins); }
    if (_sim.Citizens != _lastCitizens) { _lastCitizens = _sim.Citizens; _view.SetCitizens(_lastCitizens); }
    if (_sim.Wood != _lastWood) { _lastWood = _sim.Wood; _view.SetWood(_lastWood); }
    if ((int)_sim.Fuel != _lastFuel) { _lastFuel = (int)_sim.Fuel; _view.SetFuel(_lastFuel); }
}
void OnClaimRequested()
{
    _sim.ClaimOffline();
}
void OnFirewoodRequested()
{
    _sim.AddFirewood();
}
void OnBuildRequested()
{
    _sim.BuildWoodyard();
}
void OnDigRequested()
{
    _sim.StartDig();
}
```

```

    }
    void OnDispatchRequested(int node)
    {
        _sim.DispatchOrder(node);
    }
    void OnSocialTapped()
    {
        _view.SetHint("          —          1 / .          ");
    }
    void OnDigStageReached(int stage)
    {
        _view.SetDigStage(stage);
    }
    void OnPhaseChanged(FurnaceSim.SimPhase phase)
    {
        if (phase == FurnaceSim.SimPhase.Settled)
            _view.ShowSettle(_sim.Coins, _sim.Citizens, _sim.Wood, (int)_sim.Fuel, _sim.Day, _sim.OutsideDispatchDon
        else
            _view.HideIntro();
    }
    sealed class TelemetryDebugSink : Sy.Telemetry.TelemetryService.ISink
    {
        public void Write(string evtId, Dictionary<string, string> fields)
        {
            var sb = new System.Text.StringBuilder(evtId);
            foreach (var kv in fields)
            {
                sb.Append(' ').Append(kv.Key).Append('=').Append(kv.Value);
            }
            Debug.Log("[TELEMETRY] " + sb);
        }
    }
}

// FurnaceSim.cs
using System;
namespace Sy.P05.Newbie
{
    /// <summary>
    ///          C#          Play  Editor headless  NewbieFlowCheck
    ///          §4          ticking (fuelPerMin=4+fireLv*2· )/ lv1 / → /
    ///          ( x2+frostRisk+ )/ /
    ///          06 05 M0 →M1 (8h )+ ( 1 · +30)→M2 lv1
    /// ( + lv2· )→M3 (10s ) →M4 + ( / )
    /// →M5 + + " "( +1)
    ///          1 =
    /// </summary>
    public sealed class FurnaceSim
    {
        public enum SimPhase

```

```

{
    Intro, AwaitClaim, AwaitFirewood, AwaitBuild, AwaitDig, Digging,
    AwaitDispatch, Dispatching, Reveal, Settled
}

public SimPhase Phase { get; private set; } = SimPhase.Intro;
public float Fuel { get; private set; } = NewbieConfig.StartFuel;
public int Coins { get; private set; }
public int Citizens { get; private set; } = NewbieConfig.StartCitizens;
public int Wood { get; private set; } = NewbieConfig.StartWood;
public int Day { get; private set; } = NewbieConfig.StartDay;
public int FireLevel { get; private set; } = 1; // §4.2① →
public bool HasWoodyard { get; private set; }
public bool SurvivorFound { get; private set; }
public float PhaseTimer { get; private set; }
public float LightRadius => NewbieConfig.LightRadiusByLevel[FireLevel];
public int DigStagesSeen { get; private set; } // bit0-3=
public bool OutsideDispatchDone { get; private set; } // ×2+ +
public int AdPopupCount => 0; //
public event Action<SimPhase> PhaseChanged;
public event Action Claimed;
public event Action FirewoodAdded;
public event Action WoodyardBuilt;
public event Action FireLevelUp; // =M5
public event Action DigStarted;
public event Action<int> DigStageReached; // V1.2 ②
public event Action SurvivorDug;
public event Action<int> DispatchStarted; // ( 0 /1 )
public event Action<int, int> OrderDone; // ( , )
public event Action FrostRiskWarned; // frostRisk
public event Action Settled;
int _dispatchNode;
int _digStageDone;
float _woodTimer;
public void Tick(float dt)
{
    if (Phase == SimPhase.Intro)
    {
        PhaseTimer += dt;
        if (PhaseTimer >= NewbieConfig.IntroSeconds) SetPhase(SimPhase.AwaitClaim);
        return;
    }
    if (Phase == SimPhase.Settled) return;
    // ticking §4.1 fuelPerMin=4+fireLv*2 1
    Fuel = Math.Max(NewbieConfig.FuelFloor, Fuel - NewbieConfig.FuelDrainPerSecond(FireLevel) * dt);
    // lv1 WoodyardWoodInterval +1 WoodyardWoodCap
    if (HasWoodyard)
    {
        _woodTimer += dt;
        if (_woodTimer >= NewbieConfig.WoodyardWoodInterval && Wood < NewbieConfig.WoodyardWoodCap)
        {

```

```

        _woodTimer = 0f;
        Wood++;
    }
}
switch (Phase)
{
    case SimPhase.Digging:
        PhaseTimer += dt;
        TickDigStages();
        if (PhaseTimer >= NewbieConfig.DigSeconds)
        {
            SurvivorFound = true;
            Citizens++;
            SurvivorDug?.Invoke();
            SetPhase(SimPhase.AwaitDispatch);
        }
        break;
    case SimPhase.Dispatching:
        PhaseTimer += dt;
        if (PhaseTimer >= NewbieConfig.DispatchSeconds)
        {
            Wood += NewbieConfig.DispatchYield[_dispatchNode];
            if (_dispatchNode == 1)
            {
                OutsideDispatchDone = true;
                Fuel = Math.Max(NewbieConfig.FuelFloor, Fuel - NewbieConfig.OutsideWarmupFuel); //
            }
            OrderDone?.Invoke(NewbieConfig.DispatchYield[_dispatchNode], _dispatchNode);
            SetPhase(SimPhase.Reveal);
        }
        break;
    case SimPhase.Reveal:
        PhaseTimer += dt;
        if (PhaseTimer >= NewbieConfig.RevealSeconds)
        {
            Day++; // → +1 §4.1 +
            Settled?.Invoke();
            SetPhase(SimPhase.Settled);
        }
        break;
}
}
void TickDigStages()
{
    int stage = Math.Min(NewbieConfig.DigStages.Length,
        1 + (int)(PhaseTimer / (NewbieConfig.DigSeconds / NewbieConfig.DigStages.Length)));
    if (stage > _digStageDone)
    {
        _digStageDone = stage;
        DigStagesSeen |= 1 && (stage - 1);
    }
}

```

```

        DigStageReached?.Invoke(stage);
    }
}
/// <summary>M1a      8h      </summary>
public bool ClaimOffline()
{
    if (Phase != SimPhase.AwaitClaim) return false;
    Coins += NewbieConfig.OfflineCoins;
    Claimed?.Invoke();
    SetPhase(SimPhase.AwaitFirewood);
    return true;
}
/// <summary>M1b      1 → +30      </summary>
public bool AddFirewood()
{
    if (Phase != SimPhase.AwaitFirewood) return false;
    if (Wood < 1) return false; // 3 —
    Wood--;
    Fuel = Math.Min(100f, Fuel + NewbieConfig.FirewoodFuel);
    FirewoodAdded?.Invoke();
    SetPhase(SimPhase.AwaitBuild);
    return true;
}
/// <summary>M2      lv1      + +      lv2      ·      ① </summary>
public bool BuildWoodyard()
{
    if (Phase != SimPhase.AwaitBuild) return false;
    HasWoodyard = true;
    FireLevel++;
    FireLevelUp?.Invoke();
    WoodyardBuilt?.Invoke();
    SetPhase(SimPhase.AwaitDig);
    return true;
}
/// <summary>M3      →10s      </summary>
public bool StartDig()
{
    if (Phase != SimPhase.AwaitDig) return false;
    PhaseTimer = 0f;
    _digStageDone = 0;
    DigStarted?.Invoke();
    SetPhase(SimPhase.Digging);
    return true;
}
/// <summary>M4      0=      /1=      · x2·frostRisk+      </summary>
public bool DispatchOrder(int node)
{
    if (Phase != SimPhase.AwaitDispatch) return false;
    if (node < 0 || node >= NewbieConfig.DispatchNames.Length) return false;
    _dispatchNode = node;
}

```

```

        if (node == 1)
        {
            FrostRiskWarned?.Invoke(); // frostRisk=0.05* 15% —
        }
        PhaseTimer = 0f;
        DispatchStarted?.Invoke(node);
        SetPhase(SimPhase.Dispatching);
        return true;
    }
    void SetPhase(SimPhase p)
    {
        if (Phase == p) return;
        Phase = p;
        PhaseTimer = 0f;
        PhaseChanged?.Invoke(p);
    }
}

// FurnaceView.cs
using System;
using System.Collections;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P05.Newbie
{
    /// <summary>
    ///     UGUI— x x x x
    ///     UI Initialize      YAML      = / /      → · ① /
    ///     vs                /      / " "
    ///     =
    /// </summary>
    public sealed class FurnaceView : MonoBehaviour
    {
        Canvas _canvas;
        Sprite _square;
        FurnaceSim _sim;
        RectTransform _lightCircle, _furnaceRt;
        Image _fuelFill, _digFill, _dispatchFill;
        Image _digPortrait; // X027: dig-stage general art (frozen peek -&gt; awake reveal)
        Button _firewoodButton, _buildButton, _iceButton, _nearButton, _farButton, _socialButton;
        Image _firewoodImage, _buildImage, _iceImage, _nearImage, _farImage;
        Text _coinText, _citizenText, _woodText, _fuelText, _hintText, _digStageText;
        RectTransform _offlinePopup, _settlePanel;
        Text _settleStats;
        FurnaceSim.SimPhase _lastPhase;
        float _circleScale = NewbieConfig.LightRadiusByLevel[1];
        public event Action ClaimRequested;
        public event Action FirewoodRequested;
        public event Action BuildRequested;
        public event Action DigRequested;
    }
}

```

```

public event Action NearDispatchRequested; // .
public event Action FarDispatchRequested; // . x2.
public event Action SocialTapped;
static readonly Color SnowWhite = new Color(0.969f, 0.976f, 0.988f); // #F7F9FC
static readonly Color IceBlue = new Color(0.659f, 0.773f, 0.855f); // #A8C5DA
static readonly Color InkBlue = new Color(0.118f, 0.227f, 0.373f); // #1E3A5F
static readonly Color FurnaceOrange = new Color(1f, 0.42f, 0.102f); // #FF6B1A
static readonly Color IceGray = new Color(0.42f, 0.48f, 0.549f); // #6B7B8C
public void Initialize(Canvas canvas, FurnaceSim sim)
{
    _canvas = canvas;
    _sim = sim;
    _square = MakeSquareSprite();
    BuildScene();
    BuildHud();
    BuildOfflinePopup();
    BuildSettlePanel();
    SetHint(".....");
    StartCoroutine(SnowRoutine());
    StartCoroutine(FlameRoutine());
}
// ——
void BuildScene()
{
    var bg = MakeFullImage("Bg", InkBlue);
    bg.SetAsFirstSibling();
    // X027 generated art: swap the solid Bg block for the scene painting
    // when present; missing art keeps the procedural look untouched.
    var bgImg = bg.GetComponent<Image>();
    var bgArt = Sy.Core.ArtRuntime.Get("P05_scene_01");
    if (bgArt != null)
    {
        bgImg.sprite = bgArt;
        bgImg.color = Color.white;
        bgImg.preserveAspect = true;
        bgImg.raycastTarget = false;
    }
    //
    var ground = MakeCanvasChild("Ground");
    ground.sizeDelta = new Vector2(1080, 520);
    ground.anchoredPosition = new Vector2(0, -700);
    var groundImg = ground.GetComponent<Image>();
    groundImg.sprite = _square;
    groundImg.color = new Color(0.85f, 0.88f, 0.92f);
    // → lv1 0.55 → lv2 0.95
    _lightCircle = MakeCanvasChild("LightCircle");
    _lightCircle.sizeDelta = new Vector2(700, 700);
    _lightCircle.anchoredPosition = new Vector2(0, -420);
    var circleImg = _lightCircle.GetComponent<Image>();
    circleImg.sprite = _square;
}

```

```

circleImg.color = new Color(IceBlue.r, IceBlue.g, IceBlue.b, 0.10f);
// X027 generated art: hidden general portrait shown by dig stage below.
var portraitGo = new GameObject("DigPortrait", typeof(RectTransform), typeof(Image));
portraitGo.transform.SetParent(_canvas.transform, false);
_digPortrait = portraitGo.GetComponent<Image>();
var portraitRt = (RectTransform)portraitGo.transform;
portraitRt.anchorMin = portraitRt.anchorMax = new Vector2(0.5f, 0.5f);
portraitRt.anchoredPosition = new Vector2(0, 40);
portraitRt.sizeDelta = new Vector2(640, 860);
_digPortrait.raycastTarget = false;
_digPortrait.gameObject.SetActive(false);
//
_furnaceRt = MakeCanvasChild("Furnace");
_furnaceRt.anchoredPosition = new Vector2(0, -420);
_furnaceRt.sizeDelta = new Vector2(150, 150);
var furnaceImg = _furnaceRt.GetComponent<Image>();
furnaceImg.sprite = _square;
furnaceImg.color = FurnaceOrange;
MakeText(_furnaceRt, " ", 60, Vector2.zero, TextAnchor.MiddleCenter, 150, 150).color = InkBlue;
var fuelBar = MakeCanvasChild("FuelBar");
fuelBar.anchoredPosition = new Vector2(0, -560);
fuelBar.sizeDelta = new Vector2(300, 24);
var fuelBg = fuelBar.GetComponent<Image>();
fuelBg.sprite = _square;
fuelBg.color = new Color(0.15f, 0.18f, 0.22f);
var fillGo = new GameObject("Fill", typeof(RectTransform), typeof(Image));
fillGo.transform.SetParent(fuelBar, false);
var fillRt = (RectTransform)fillGo.transform;
fillRt.anchorMin = new Vector2(0f, 0.5f);
fillRt.anchorMax = new Vector2(1f, 0.5f);
fillRt.anchoredPosition = Vector2.zero;
fillRt.sizeDelta = new Vector2(0, 24);
_fuelFill = fillGo.GetComponent<Image>();
_fuelFill.sprite = _square;
_fuelFill.type = Image.Type.Filled;
_fuelFill.fillMethod = Image.FillMethod.Horizontal;
_fuelFill.color = FurnaceOrange;
//
_firewoodButton = MakeActionButton("FirewoodButton", " ", new Vector2(-260, -420), FirewoodRequested, out _
//
_buildButton = MakeActionButton("BuildButton", " \n ", new Vector2(300, -420), BuildRequested, out _buildIm
//
_iceButton = MakeActionButton("IceButton", " \n ", new Vector2(-300, -660), DigRequested, out _iceImage);
//      M4      /      . x2.  —
_nearButton = MakeActionButton("NearNode", " . \n+6 ", new Vector2(170, -470), NearDispatchRequested, ou
_farButton = MakeActionButton("FarNode", " . \n+12 15%", new Vector2(450, -260), FarDispatchRequested, o
// X027 generated art: furnace + building paintings on the action blocks.
ApplyActionArt(_furnaceRt.GetComponent<Image>(), "P05_icon_fire");
ApplyActionArt(_firewoodImage, "P05_icon_wood");
ApplyActionArt(_buildImage, "P05_building_01_wood");

```

```
ApplyActionArt(_iceImage, "P05_building_02_ice");
ApplyActionArt(_nearImage, "P05_building_03_farm");
ApplyActionArt(_farImage, "P05_building_04_iron");
//
var digBar = MakeCanvasChild("DigBar");
digBar.anchoredPosition = new Vector2(-300, -540);
digBar.sizeDelta = new Vector2(220, 18);
var digBg = digBar.GetComponent<Image>();
digBg.sprite = _square;
digBg.color = new Color(0.15f, 0.18f, 0.22f);
var digFillGo = new GameObject("Fill", typeof(RectTransform), typeof(Image));
digFillGo.transform.SetParent(digBar, false);
var digFillRt = (RectTransform)digFillGo.transform;
digFillRt.anchorMin = new Vector2(0f, 0.5f);
digFillRt.anchorMax = new Vector2(1f, 0.5f);
digFillRt.anchoredPosition = Vector2.zero;
digFillRt.sizeDelta = new Vector2(0, 18);
_digFill = digFillGo.GetComponent<Image>();
_digFill.sprite = _square;
_digFill.type = Image.Type.Filled;
_digFill.fillMethod = Image.FillMethod.Horizontal;
_digFill.color = IceBlue;
digBar.gameObject.SetActive(false);
//      "      "
_digStageText = MakeText(_canvas.transform, "", 34, new Vector2(-300, -450), TextAnchor.MiddleCenter, 340, 6
_digStageText.color = new Color(0.05f, 0.09f, 0.15f); // U184: deep-ink - was IceBlue-on-snow 1.4:1 (17 do
_digStageText.gameObject.SetActive(false);
//
var dispatchBar = MakeCanvasChild("DispatchBar");
dispatchBar.anchoredPosition = new Vector2(310, -100);
dispatchBar.sizeDelta = new Vector2(220, 18);
var dispBg = dispatchBar.GetComponent<Image>();
dispBg.sprite = _square;
dispBg.color = new Color(0.15f, 0.18f, 0.22f);
var dispFillGo = new GameObject("Fill", typeof(RectTransform), typeof(Image));
dispFillGo.transform.SetParent(dispatchBar, false);
var dispFillRt = (RectTransform)dispFillGo.transform;
dispFillRt.anchorMin = new Vector2(0f, 0.5f);
dispFillRt.anchorMax = new Vector2(1f, 0.5f);
dispFillRt.anchoredPosition = Vector2.zero;
dispFillRt.sizeDelta = new Vector2(0, 18);
_dispatchFill = dispFillGo.GetComponent<Image>();
_dispatchFill.sprite = _square;
_dispatchFill.type = Image.Type.Filled;
_dispatchFill.fillMethod = Image.FillMethod.Horizontal;
_dispatchFill.color = IceBlue;
dispatchBar.gameObject.SetActive(false);
//      M5
_socialButton = MakeActionButton("SocialButton", "      \n      ", new Vector2(380, 300), SocialTapped, out _social
_socialButton.gameObject.SetActive(false);
```

```

}
Image _socialImage;
void BuildHud()
{
    var root = _canvas.transform;
    _coinText = MakeText(root, "  0", 46, new Vector2(0, 880), TextAnchor.UpperLeft, 360, 70);
    _coinText.color = SnowWhite;
    _fuelText = MakeText(root, "  20%", 46, new Vector2(0, 880), TextAnchor.UpperCenter, 380, 70);
    _fuelText.color = FurnaceOrange;
    _citizenText = MakeText(root, "  1", 44, new Vector2(0, 820), TextAnchor.UpperRight, 320, 60);
    _citizenText.color = SnowWhite;
    _woodText = MakeText(root, "  3", 44, new Vector2(0, 760), TextAnchor.UpperRight, 320, 60);
    _woodText.color = SnowWhite; // U184: was IceGray-on-deep ~2.6:1 (17 doc section 3.7)
    _hintText = MakeText(root, "", 40, new Vector2(0, -840), TextAnchor.MiddleCenter, 1020, 150);
    _hintText.color = new Color(0.05f, 0.09f, 0.15f); // U184: deep-ink on snow band - was SnowWhite-on-snow 1
}
void BuildOfflinePopup()
{
    _offlinePopup = MakeOverlayPanel("OfflinePopup", new Color(0.06f, 0.10f, 0.16f, 0.94f));
    MakeText(_offlinePopup, "  8  ", 66, new Vector2(0, 220), TextAnchor.MiddleCenter, 900, 100).color = SnowW
    MakeText(_offlinePopup, "  x240  ", 52, new Vector2(0, 80), TextAnchor.MiddleCenter, 900, 90).color = Ice
    MakePanelButton(_offlinePopup, "  ", new Vector2(0, -160), () => ClaimRequested?.Invoke());
}
void BuildSettlePanel()
{
    _settlePanel = MakeOverlayPanel("SettlePanel", new Color(0.06f, 0.10f, 0.16f, 0.96f));
    MakeText(_settlePanel, "  ", 96, new Vector2(0, 560), TextAnchor.MiddleCenter, 1040, 120).color = Furnac
    _settleStats = MakeText(_settlePanel, "", 44, new Vector2(0, 360), TextAnchor.MiddleCenter, 1020, 380);
    MakePanelButton(_settlePanel, "  ", new Vector2(-260, -300), () => UnityEngine.SceneManagement.SceneMan
    MakePanelButton(_settlePanel, "  ", new Vector2(260, -300), () => UnityEngine.SceneManagement.SceneMan
    _settlePanel.gameObject.SetActive(false);
}
// _____
public void SyncView(FurnaceSim sim)
{
    var phase = sim.Phase;
    if (phase != _lastPhase)
    {
        _lastPhase = phase;
        OnPhaseVisual(phase);
    }
    float fuel01 = sim.Fuel / 100f;
    _fuelFill.fillAmount = fuel01;
    //      →      ① +
    float target = NewbieConfig.LightRadiusByLevel[sim.FireLevel] + fuel01 * 0.05f;
    _circleScale = Mathf.Lerp(_circleScale, target, 0.08f);
    _lightCircle.localScale = Vector3.one * _circleScale;
    if (phase == FurnaceSim.SimPhase.Digging)
        _digFill.fillAmount = sim.PhaseTimer / NewbieConfig.DigSeconds;
    if (phase == FurnaceSim.SimPhase.Dispatching)

```

```

        _dispatchFill.fillAmount = sim.PhaseTimer / NewbieConfig.DispatchSeconds;
        SetButtonState(_firewoodButton, _firewoodImage, phase == FurnaceSim.SimPhase.AwaitFirewood);
        SetButtonState(_buildButton, _buildImage, phase == FurnaceSim.SimPhase.AwaitBuild);
        SetButtonState(_iceButton, _iceImage, phase == FurnaceSim.SimPhase.AwaitDig);
        bool dispatchable = phase == FurnaceSim.SimPhase.AwaitDispatch;
        SetButtonState(_nearButton, _nearImage, dispatchable);
        SetButtonState(_farButton, _farImage, dispatchable);
    }
    void OnPhaseVisual(FurnaceSim.SimPhase phase)
    {
        switch (phase)
        {
            case FurnaceSim.SimPhase.AwaitClaim:
                _offlinePopup.gameObject.SetActive(true);
                break;
            case FurnaceSim.SimPhase.AwaitFirewood:
                _offlinePopup.gameObject.SetActive(false);
                break;
            case FurnaceSim.SimPhase.Digging:
                _digFill.transform.parent.gameObject.SetActive(true);
                _digStageText.gameObject.SetActive(true);
                break;
            case FurnaceSim.SimPhase.AwaitDispatch:
                _digFill.transform.parent.gameObject.SetActive(false);
                _digStageText.gameObject.SetActive(false);
                break;
            case FurnaceSim.SimPhase.Dispatching:
                _dispatchFill.transform.parent.gameObject.SetActive(true);
                break;
            case FurnaceSim.SimPhase.Reveal:
                _dispatchFill.transform.parent.gameObject.SetActive(false);
                _socialButton.gameObject.SetActive(true);
                break;
            case FurnaceSim.SimPhase.Settled:
                _socialButton.gameObject.SetActive(true);
                break;
        }
    }
}

static void SetButtonState(Button button, Image image, bool active)
{
    if (button.interactable == active) return;
    button.interactable = active;
    image.color = active ? FurnaceOrange : new Color(0.30f, 0.34f, 0.40f);
}

// —— HUD ——

public void SetCoins(int c) => _coinText.text = "    " + c;
public void SetCitizens(int n) => _citizenText.text = "    " + n;
public void SetWood(int w) => _woodText.text = "    " + w;
public void SetFuel(int f) => _fuelText.text = "    " + f + "%";
public void SetHint(string hint) => _hintText.text = hint;

```

```

/// <summary>          → → → </summary>
public void SetDigStage(int stage)
{
    _digStageText.text = NewbieConfig.DigStages[stage - 1] + " " + stage * 25 + "% ";
    // X027: frozen silhouette peek at "reveal", full-color awake at "awake".
    if (_digPortrait == null) return;
    var art = stage >= 4 ? Sy.Core.ArtRuntime.Get("P05_general_02_guanyu_awake")
        : stage == 3 ? Sy.Core.ArtRuntime.Get("P05_general_02_guanyu_frozen")
        : null;
    if (art != null)
    {
        _digPortrait.sprite = art;
        _digPortrait.color = stage >= 4 ? Color.white : new Color(1f, 1f, 1f, 0.6f);
        _digPortrait.preserveAspect = true;
        _digPortrait.gameObject.SetActive(true);
    }
    else
    {
        _digPortrait.gameObject.SetActive(false);
    }
}
public void HideIntro()
{
    // M0 Hint
}
public void ShowSettle(int coins, int citizens, int wood, int fuel, int day, bool outside)
{
    _settlePanel.gameObject.SetActive(true);
    _settleStats.text = $" {coins} · {citizens} · {wood} · {fuel}%\n {day} {NewbieConfig.Miles}
}
// ——
/// <summary>X027: swap a solid action block for generated art when the key exists.</summary>
static void ApplyActionArt(Image image, string key)
{
    var art = Sy.Core.ArtRuntime.Get(key);
    if (image == null || art == null) return;
    image.sprite = art;
    image.color = Color.white;
    image.preserveAspect = true;
}
IEnumerator SnowRoutine()
{
    var flakes = new RectTransform[7];
    for (int i = 0; i < flakes.Length; i++)
    {
        var go = new GameObject("Snow_" + i, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_canvas.transform, false);
        flakes[i] = (RectTransform)go.transform;
        flakes[i].anchorMin = flakes[i].anchorMax = new Vector2(0.5f, 0.5f);
        flakes[i].sizeDelta = new Vector2(18, 18);
    }
}

```

```
        flakes[i].anchoredPosition = new Vector2(UnityEngine.Random.Range(-520f, 520f), UnityEngine.Random.Range(-520f, 520f));
        var img = go.GetComponent<Image>();
        img.sprite = _square;
        img.color = new Color(SnowWhite.r, SnowWhite.g, SnowWhite.b, 0.75f);
        flakes[i].SetAsFirstSibling();
        flakes[i].SetSiblingIndex(1);
    }
    while (true)
    {
        for (int i = 0; i < flakes.Length; i++)
        {
            var pos = flakes[i].anchoredPosition;
            pos.y -= 60f * Time.deltaTime;
            pos.x += Mathf.Sin(Time.time + i) * 10f * Time.deltaTime;
            if (pos.y < -950f)
            {
                pos.y = 950f;
                pos.x = UnityEngine.Random.Range(-520f, 520f);
            }
            flakes[i].anchoredPosition = pos;
        }
        yield return null;
    }
}

IEnumerator FlameRoutine()
{
    while (true)
    {
        float fuel01 = _sim.Fuel / 100f;
        float pulse = 0.5f + 0.5f * Mathf.Sin(Time.unscaledTime * (2f + fuel01 * 3f));
        _furnaceRt.localScale = Vector3.one * (1f + 0.05f + 0.06f * pulse * fuel01);
        yield return null;
    }
}

// -----
RectTransform MakeCanvasChild(string name)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    return rt;
}

RectTransform MakeFullImage(string name, Color color)
{
    var rt = MakeCanvasChild(name);
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.anchoredPosition = Vector2.zero;
    rt.sizeDelta = Vector2.zero;
}
```

```
        rt.GetComponent<Image>().color = color;
        return rt;
    }
    Button MakeActionButton(string name, string label, Vector2 pos, Action onClick, out Image image)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(160, 160);
        image = go.GetComponent<Image>();
        image.sprite = _square;
        image.color = new Color(0.30f, 0.34f, 0.40f);
        var label1 = MakeText(go.transform, label, 34, Vector2.zero, TextAnchor.MiddleCenter, 170, 170);
        label1.color = SnowWhite;
        var button = go.GetComponent<Button>();
        button.interactable = false;
        button.onClick.AddListener(() => onClick());
        return button;
    }
    Button MakePanelButton(RectTransform parent, string label, Vector2 pos, Action onClick)
    {
        var go = new GameObject(label, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(420, 120);
        var image = go.GetComponent<Image>();
        image.sprite = _square;
        image.color = FurnaceOrange;
        MakeText(go.transform, label, 48, Vector2.zero, TextAnchor.MiddleCenter, 420, 120).color = InkBlue;
        var button = go.GetComponent<Button>();
        button.onClick.AddListener(() => onClick());
        return button;
    }
    RectTransform MakeOverlayPanel(string name, Color color)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
        var image = go.GetComponent<Image>();
        image.sprite = _square;
        image.color = color;
        return rt;
    }
}
```

```

static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
{
    var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(w, h);
    var t = go.GetComponent<Text>();
    t.font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
    t.text = text;
    t.fontSize = size;
    t.alignment = anchor;
    t.color = new Color(0.91f, 0.89f, 0.85f);
    t.supportRichText = true;
    return t;
}

static Sprite MakeSquareSprite()
{
    var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
    var px = tex.GetPixels32();
    for (int i = 0; i < px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
    tex.SetPixels32(px);
    tex.Apply(false, false);
    return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
}
}

// NewbieConfig.cs
namespace Sy.P05.Newbie
{
    /// <summary>
    /// P05 . V0.2 — X009+IConfigService Design/configs/P05 JSON
    /// V1.3§4.1 ticking (fuelPerMin="4+fireLv*2"· )/§4.2 ① = ( → /
    /// x2·frostRisk="0.05*distFromFire")/② ( → → → —"4h " " " )/
    /// D1-D2 / + 06 05 M0-M5
    /// #F7F9FC × #A8C5DA × #1E3A5F × #FF6B1A × #6B7B8C
    /// 10 →10 →10s §4.2
    /// </summary>
    public static class NewbieConfig
    {
        public const string GameTitle = " ";
        public const string GameSlogan = " ";
        public const float IntroSeconds = 3f; // M0 +
        public const float DigSeconds = 10f; // M3 10 ·
        public const float DispatchSeconds = 10f; // M4
        public const float RevealSeconds = 2f; // M5
        public const float MaxTutorialSeconds = 210f; // ≤3.5min 06 ②
        // — §4.1 fuelPerMin="4+fireLv*2" —
        public const float StartFuel = 20f; // M0
    }
}

```

```

    public const float FuelFloor = 1f; //
    public const float FirewoodFuel = 30f; // M1 +30 1
    public const int StartWood = 3; //
    public const float WoodyardWoodInterval = 3f; // lv1 3s+1
    public const int WoodyardWoodCap = 5; //
    /// <summary> fuelPerMin="4+fireLv*2" = /60 </summary>
    public static float FuelDrainPerSecond(int fireLevel) => (4f + fireLevel * 2f) / 60f;
    // — = §4.2 ① → = —
    public static readonly float[] LightRadiusByLevel = { 0f, 0.55f, 0.95f }; // [lv1][lv2]
    /// <summary> 0= 1= . ×2 </summary>
    public static readonly string[] DispatchNames = { " . ", " . " };
    public static readonly int[] DispatchYield = { 6, 12 }; // ×2
    public static readonly float[] DispatchFrostRisk = { 0f, 0.15f }; // frostRisk=0.05* 15%
    public const float OutsideWarmupFuel = 5f; // —
    // — V1.2 ② + " " —
    public static readonly string[] DigStages =
    {
        " — ",
        " — ",
        " — ",
        " — ",
    };
};
// — §4.1 + D1-D2 —
public const int StartDay = 1;
public const string ColdWaveTeaser = " —D1-D2 D3 ";
public const int MilestoneDay = 7; // 7
public const int OfflineCoins = 240; // M1 8h
public const int StartCitizens = 1; // →2
}
}
// TutorialFlow.cs
using System;
namespace Sy.P05.Newbie
{
    /// <summary>
    /// evt_newbie_step 1-5 ① + ② ③ ④ + ⑤ +
    /// FurnaceSim / / —
    /// Play headless NewbieFlowCheck
    /// </summary>
    public sealed class TutorialFlow
    {
        public int Step { get; private set; }
        readonly Action<string> _hint;
        readonly Action<int> _onStep;
        public TutorialFlow(FurnaceSim sim, Action<string> hint, Action<int> onStep)
        {
            _hint = hint ?? delegate { };
            _onStep = onStep ?? delegate { };
            sim.PhaseChanged += OnPhaseChanged;
            sim.Claimed += () => _hint(" 8h . — 1 . +30 ");
        }
    }
}

```

```

sim.FirewoodAdded += OnFirewoodAdded;
sim.WoodyardBuilt += OnWoodyardBuilt;
sim.FireLevelUp += OnFireLevelUp;
sim.DigStarted += () => _hint(" ..... 10 . ");
sim.DigStageReached += OnDigStageReached;
sim.SurvivorDug += OnSurvivorDug;
sim.DispatchStarted += OnDispatchStarted;
sim.FrostRiskWarned += () => _hint(" . 15% frostRisk — 5 x2");
sim.OrderDone += OnOrderDone;
sim.Settled += OnSettled;
_hint(" — ");
}
void Advance(int step)
{
    if (Step >= step) return;
    Step = step;
    _onStep(step);
}
void OnPhaseChanged(FurnaceSim.SimPhase phase)
{
    switch (phase)
    {
        case FurnaceSim.SimPhase.AwaitClaim:
            _hint(" 8 — ");
            break;
        case FurnaceSim.SimPhase.AwaitFirewood:
            _hint(" — +30");
            break;
        case FurnaceSim.SimPhase.AwaitBuild:
            _hint(" — lv1 . . lv2 ");
            break;
        case FurnaceSim.SimPhase.AwaitDig:
            _hint(" — ");
            break;
        case FurnaceSim.SimPhase.AwaitDispatch:
            _hint(" +1 x2");
            break;
        case FurnaceSim.SimPhase.Reveal:
            _hint(" — " + NewbieConfig.ColdWaveTeaser + " ");
            break;
    }
}
void OnFirewoodAdded()
{
    Advance(1);
    _hint(" — + ");
}
void OnWoodyardBuilt()
{
    Advance(2);
}

```

```

        _hint(" lv1          3s+1 · 5 — ");
    }
    void OnFireLevelUp()
    {
        _hint(" lv2 — ③ → ");
    }
    void OnDigStageReached(int stage)
    {
        _hint(NewbieConfig.DigStages[stage - 1] + " " + stage * 25 + "% ");
    }
    void OnSurvivorDug()
    {
        Advance(3);
        _hint(" \ " \"— ·4-24h ");
    }
    void OnDispatchStarted(int node)
    {
        Advance(4);
        _hint(" — " + (node == 1 ? " " : " ") + " ");
    }
    void OnOrderDone(int yield, int node)
    {
        _hint(" +" + yield + (node == 1 ? " ×2· 5 " : " ") + "— ");
    }
    void OnSettled()
    {
        Advance(5);
        _hint(" " + NewbieConfig.StartDay + " → " + (NewbieConfig.StartDay + 1)
            + " — " + NewbieConfig.MilestoneDay + " ");
    }
}
}

```